13

MCP: conectando o agent às suas ferramentas

Sem acesso estruturado às ferramentas do projeto — banco, browser, APIs — o agent improvisa: gera scripts soltos, monta comandos bash frágeis, pede sua ajuda a cada passo. O MCP resolve isso com um protocolo único: qualquer ferramenta que o implemente funciona em qualquer agent que o suporte. Este material mostra como usar servidores prontos e como criar o do seu projeto.

A IDEIA CENTRAL DESTE MATERIAL

Para o agent, uma tool de MCP é indistinguível de Read ou Write: ele a descobre, lê a descrição e decide quando chamar. A diferença entre "execute este script SQL" e "consulte o banco com esta query tipada" é a diferença entre um agent que improvisa e um que opera com precisão.

1 O que é o MCP

MCP (Model Context Protocol) é um padrão aberto que conecta agents de IA a ferramentas externas por uma interface unificada. Criado pela Anthropic no fim de 2024, foi adotado pelos grandes provedores em 2025 e doado à Linux Foundation — consolidando-se como o protocolo universal entre agents e ferramentas.

Antes dele, cada combinação agent × ferramenta exigia integração customizada. Com ele, um servidor expõe **tools tipadas** que o agent descobre no startup e chama quando julga relevante:

```
Startup → agent descobre as tools do servidor
          (nome, descrição, parâmetros tipados)
Você faz uma requisição
       → agent decide: preciso de uma tool do MCP?
       → chama com os parâmetros corretos
       → servidor executa e retorna
       → agent usa o resultado e continua
```

Três transportes: **stdio** (local, processo filho), **HTTP/SSE** (remoto) e **in-process** (embarcado no agent).

2 Usando um MCP pronto: Playwright

O exemplo mais didático — o resultado é visível na hora: o agent controla um browser de verdade.

```
# Claude Code
claude mcp add playwright -- npx @playwright/mcp@latest
/mcp   → playwright ✓ connected

# OpenCode — opencode.json
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "playwright": {
      "type": "local",
      "command": ["npx", "@playwright/mcp@latest"],
      "enabled": true
    }
  }
}
```

Com isso o agent ganha `browser_navigate`, `browser_click`, `browser_fill`, `browser_screenshot`, `browser_snapshot` — sem você escrever uma linha de automação.

3 Criando o MCP do projeto

Antes de escrever código, deixe o próprio agent propor — ele conhece a SPEC:

▸ PROMPT PARA DESCOBRIR QUAL MCP CRIAR

```
Leia o SPEC.md e o PLAN.md na raiz do projeto.

Com base nos módulos e integrações definidos, identifique
qual ferramenta externa mais beneficiaria o projeto via MCP.

Considere:
- Quais ferramentas externas o projeto usa?
- Quais consultas/operações se repetem e poderiam virar tools?
- O que o agent não consegue fazer hoje que um MCP resolveria?

Proponha 3 opções de MCP com as tools que cada um exporia
e o problema que resolve.

Após minha escolha, crie o servidor completo em
.mcp/nome-do-servidor/ com index.ts, README.md de instalação
e configuração para Claude Code e OpenCode.
```

Setup e estrutura

```
mkdir -p .mcp/meu-servidor && cd .mcp/meu-servidor
npm init -y
npm install @modelcontextprotocol/sdk zod
npm install -D typescript ts-node @types/node

.mcp/meu-servidor/
├── index.ts        ← servidor e tools
├── package.json
├── tsconfig.json
└── README.md       ← instruções de setup
```

Esqueleto do servidor

```
// .mcp/meu-servidor/index.ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js"
import { StdioServerTransport } from
  "@modelcontextprotocol/sdk/server/stdio.js"
import { z } from "zod"

const server = new McpServer({
  name: "meu-servidor",
  version: "1.0.0"
})

server.tool(
  "nome-da-tool",
  "Descrição clara do que faz e quando usar",
  {
    parametro: z.string().describe("O que este parâmetro representa")
  },
  async ({ parametro }) => {
    try {
      const resultado = await fazerAlgo(parametro)
      return { content: [{ type: "text",
               text: JSON.stringify(resultado, null, 2) }] }
    } catch (error) {
      return { content: [{ type: "text",
               text: `Erro: ${error.message}` }],
               isError: true }   ← nunca deixe exceção derrubar o servidor
    }
  }
)

const transport = new StdioServerTransport()
await server.connect(transport)
```

> **As três regras de uma boa tool:** nome é verbo no infinitivo (`consultar_pedidos`, `buscar_usuario`); a *description* é o que o agent lê para decidir quando usar — escreva como explicaria a um colega; todo parâmetro tem `.describe()` dizendo o que representa no contexto do projeto.

4 Registrando e testando

```
# Claude Code — via CLI
claude mcp add meu-servidor -- npx ts-node .mcp/meu-servidor/index.ts
/mcp   → meu-servidor ✓ connected

# ou em .claude/settings.json
{ "mcpServers": { "meu-servidor": {
    "command": "npx",
    "args": ["ts-node", ".mcp/meu-servidor/index.ts"] } } }

# OpenCode — opencode.json
{ "mcp": { "meu-servidor": {
    "type": "local",
    "command": ["npx", "ts-node", ".mcp/meu-servidor/index.ts"],
    "enabled": true } } }
```

Dois testes de aceitação, direto no chat:

1. *"Liste as tools disponíveis no MCP que acabamos de criar"* — o agent deve listá-las com nome, descrição e parâmetros, exatamente como definidas.
2. *"Use as tools do MCP para resolver uma tarefa real do projeto sem escrever código adicional"* — o agent deve escolher a tool certa, chamá-la com os parâmetros corretos e usar o resultado.

5 Versionando: o MCP é arquitetura

Um MCP que só existe na máquina de quem criou não faz parte do projeto. Commite `.mcp/` completo (com `package.json`) e a configuração de registro (`settings.json` / `opencode.json`). O README do servidor traz os comandos exatos de instalação — quem clona configura sem intervenção manual, o mesmo princípio de entrega do material 12.

6 Erros mais comuns

ERRO	CORREÇÃO
Tool sem description útil ("faz uma consulta")	A description é o critério de decisão do agent. "Consulta registros pelo identificador e retorna dados completos com status e histórico" — isso decide.
Parâmetro sem `.describe()`	O Zod aceita, mas o agent não sabe o que passar. Todo parâmetro descrito, sempre.

Agent não encontra as tools	Registro incorreto ou servidor fora do ar. Verifique com `/mcp` (Claude Code) ou perguntando "quais MCPs estão disponíveis?" (OpenCode).
Exceção sem try/catch no handler	Derruba o servidor — e o agent perde **todas** as tools. Sempre retorne `isError: true` com a mensagem.
MCP fora do git / dependências sem instrução	`.mcp/` versionado + README com `npm install` no setup. Sem isso, o servidor falha no clone.

▸ COLA PARA FIXAR

- MCP = protocolo **universal** entre agents e ferramentas: um servidor, tools tipadas, qualquer agent compatível.
- Para o agent, tool de MCP é como Read/Write: **descoberta no startup, decidida pela description**.
- Comece usando um pronto (Playwright) — o valor fica visível em minutos.
- Boa tool: **verbo no nome, description de colega, .describe() em todo parâmetro**.
- Handler sem try/catch derruba o servidor inteiro: **isError: true** sempre.
- MCP é arquitetura: **versionado** junto de agents, skills e hooks — configurável do zero após o clone.